" OrbitMind AI– Intelligent Productivity & Study Companion"

by

Md. Imroz Newaz

CSE 11220321052

Zeniya Naznin

CSE 11230121138

Fatema Parvin Kanta

CSE 11230121192

Md. Mahfuzur Rahman Hridoy

CSE 11230121186

WEEK 09 ASSIGNMENT

Course Title: Software Development III: AI-Based Software Project Development

Course Code: CSE 4104

Section: 7D | Spring 2026

Submitted to

Md. Riaz Mahmud

Assistant Professor

Department of Computer Science and Engineering

Northern University of Business and Technology, Khulna

Khulna 9100, Bangladesh

CHAPTER I

1.1 Team Information Sheet

Field	Details
Team Name	CSE4104-7D-T04
Section	7D
Project Title	OrbitMind AI — Smart Productivity & Study Assistant
Team Leader	Md. Imroz Newaz (ID: 11220321052)

1.1.1 Team Members

Member Name	Student ID	Role
Md. Imroz Newaz	11220321052	Team Leader & Database Manager
Zeniya Naznin	11230121138	Frontend Developer
Fatema Parvin Kanta	11230121192	Backend Developer
Md. Mahfuzur Rahman Hridoy	11230121186	AI Integration Lead

CHAPTER II

1. Summary of Completed Features We have successfully transitioned from isolated modular development into a cohesive, fully integrated software system. The following core features promised in our original SRS are now 100% complete:

Central Dashboard: Fully integrated. Dynamically renders workload statistics and burnout risks directly from the MongoDB database. Task Management: Fully integrated. Users can execute complete CRUD operations (Create, Read, Update, Delete) on tasks.

AI Coach Chatbot: Fully integrated. Features intent-recognition to automatically schedule tasks via natural language.

AI Study Planner: Fully integrated. Generates customized day-by-day JSON roadmaps via the Gemini API.

AI Note Summarizer: Fully integrated. Condenses academic text into key concepts.

Secure Authentication: Fully integrated. Supports Registration, Login, and Logout with stateless JWT session guards.

2. Partially Completed & Remaining Features

Partially Completed: Real-time push notifications. Currently, the notification dropdown dynamically renders alerts on page-load, but WebSocket (Socket.io) integration for instant live-push alerts is still undergoing optimization.

Remaining Features: Final system debugging, stress testing, and production deployment configuration (e.g., deploying the Node.js backend to Render/Railway).

3. Frontend–Backend Integration Status

The frontend and backend are completely synced. We eradicated all hardcoded static data from our UI. Our custom apiFetch() wrapper ensures that every frontend component correctly sends authenticated HTTP requests to the Express.js routing layer. Form submissions (e.g., Registration, Task Creation) correctly insert data into the MongoDB Atlas cluster, and the UI immediately re-renders the updated state.

4. AI Integration Status

AI is seamlessly connected to the actual application workflow. Complex user constraints are gathered via the frontend UI, routed securely to the backend controllers, and injected into strict System Prompts before hitting the Google Gemini API. The AI generates JSON outputs which the backend parses, saves to the database, and sends to the frontend for structured display.

5. Authentication Status

We have a fully functional JWT (JSON Web Token) authentication flow. Passwords are encrypted using bcryptjs before database insertion. Protected routes are secured by a middleware guard (checkAuth()) on the frontend, ensuring unauthorized users are immediately redirected to the Login page.

6. Major Problems Encountered & Solutions Implemented

Problem: The Gemini API would occasionally timeout or return unstructured markdown text, causing frontend crashes when parsing JSON.parse().
Solution: We implemented a highly resilient geminiService.js architecture. It features a failover loop (MODELS_TO_TRY) that gracefully steps down to alternate models (e.g., gemini-1.5-flash) if the primary API fails. Furthermore, we integrated a Regex stripping function (cleanJSONString) to sanitize AI responses before parsing.

Problem: Form submission layout breaks on smaller screens.
Solution: We audited our CSS grid variables and established strict mobile-first media queries, ensuring responsive fluidity on mobile and tablet viewports.

7. Changes from Original Plan

There are no major deviations from our approved Week 05 System Design. The scope remains identical to our original proposal, ensuring all critical productivity features are delivered as promised.

8.Feature Completion Checklist

Feature	Status	Reason / Planned Completion Date
JWT Authentication & Routing	**Completed**	Operational.
Dashboard & Workload Engine	**Completed**	Operational.
CRUD Task Manager	**Completed**	Operational.
AI Coach Chatbot	**Completed**	Operational.
AI Study Plan Generator	**Completed**	Operational.
AI Note Summarizer	**Completed**	Operational.
Mobile Responsiveness	**Completed**	Operational.
Live Push Notifications	**In Progress**	Finalizing Socket.io state limits. Target: Week 10.
Production Deployment	**Not Started**	Reserved for final phase. Target: Week 11.

9. Repository & Deployment Links

Official GitHub Repo: https://github.com/Mahfuz-Hridoy/cse4104-7d-t04-orbitmind-ai.git
Live Demo Link: Localhost deployment for Mid-Project Review

CHAPTER III

SCREENSHOTS:

Login/Registration (Authentication Workflow)

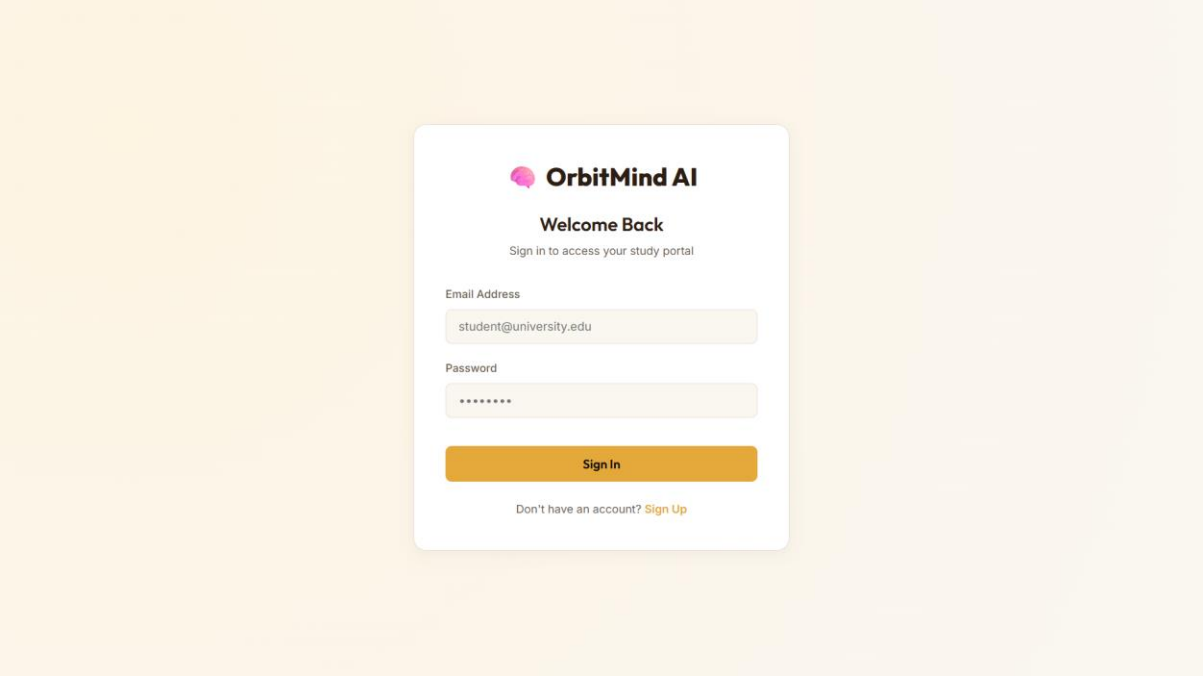


Figure 1: The secure authentication interface successfully guarding access to the system.

Central Dashboard (Frontend-Backend Integration)

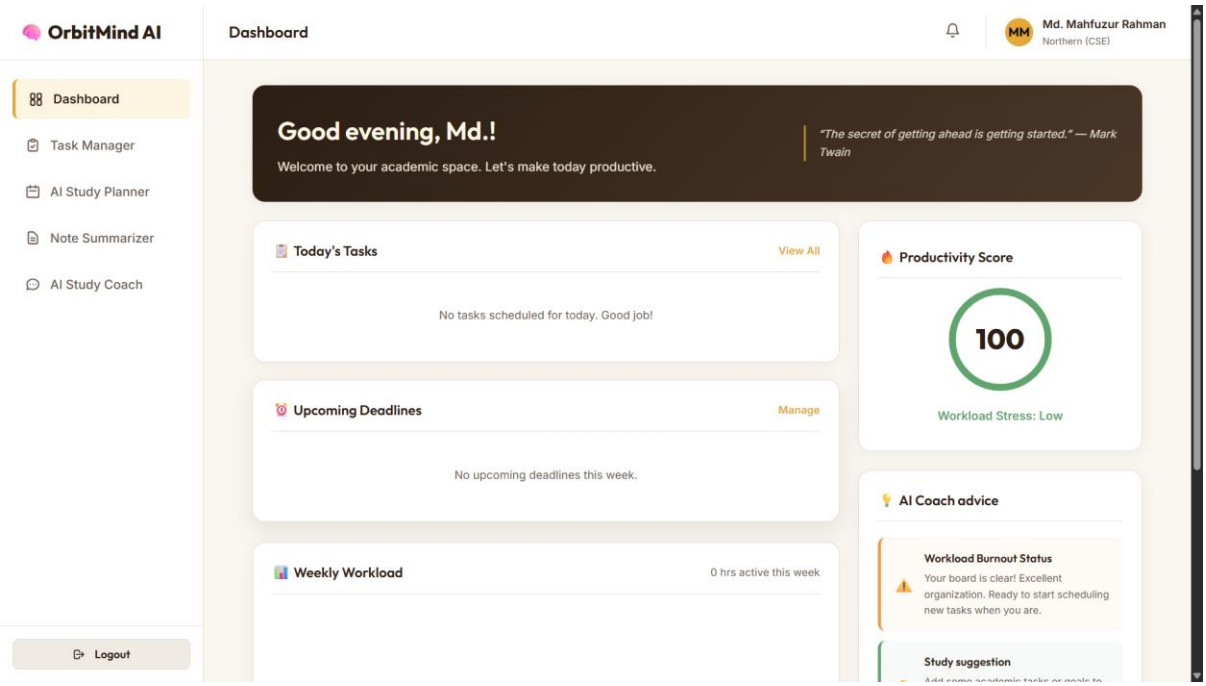


Figure 2: The dashboard dynamically loading and rendering real user data directly from the MongoDB Atlas database.

Major Functional Module (Task Manager)

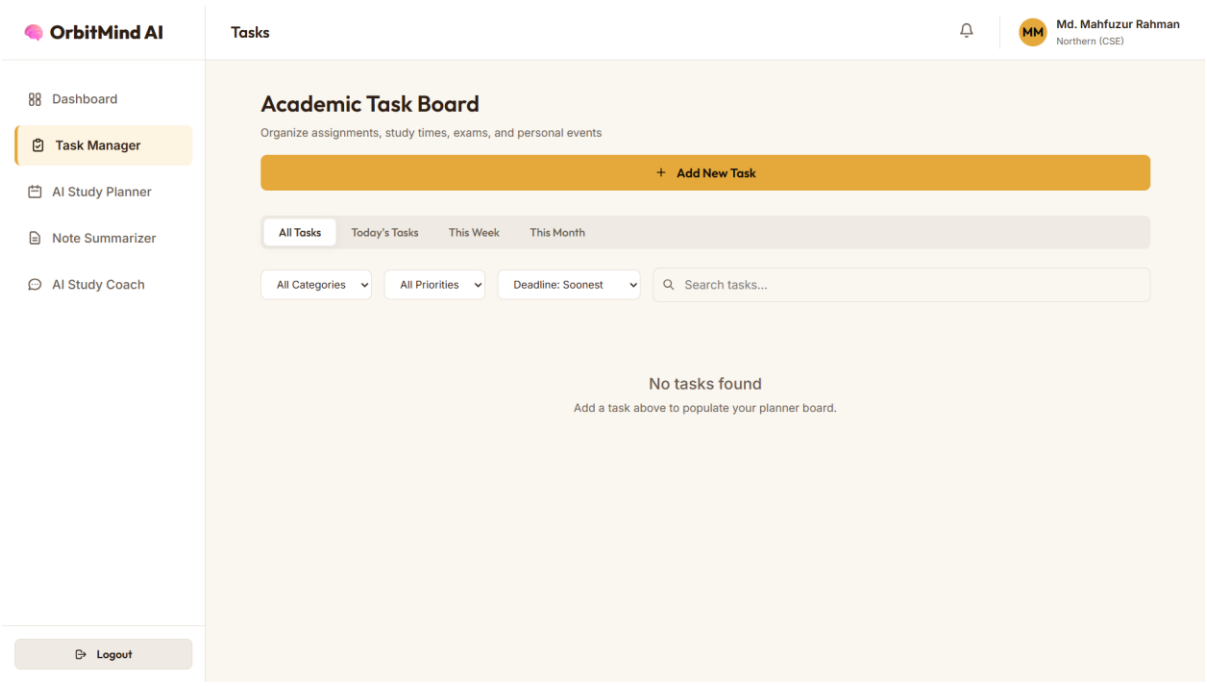


Figure 3: The interactive Task Manager demonstrating successful end-to-end CRUD operations.

AI Feature & Important User Workflow (Study Planner)

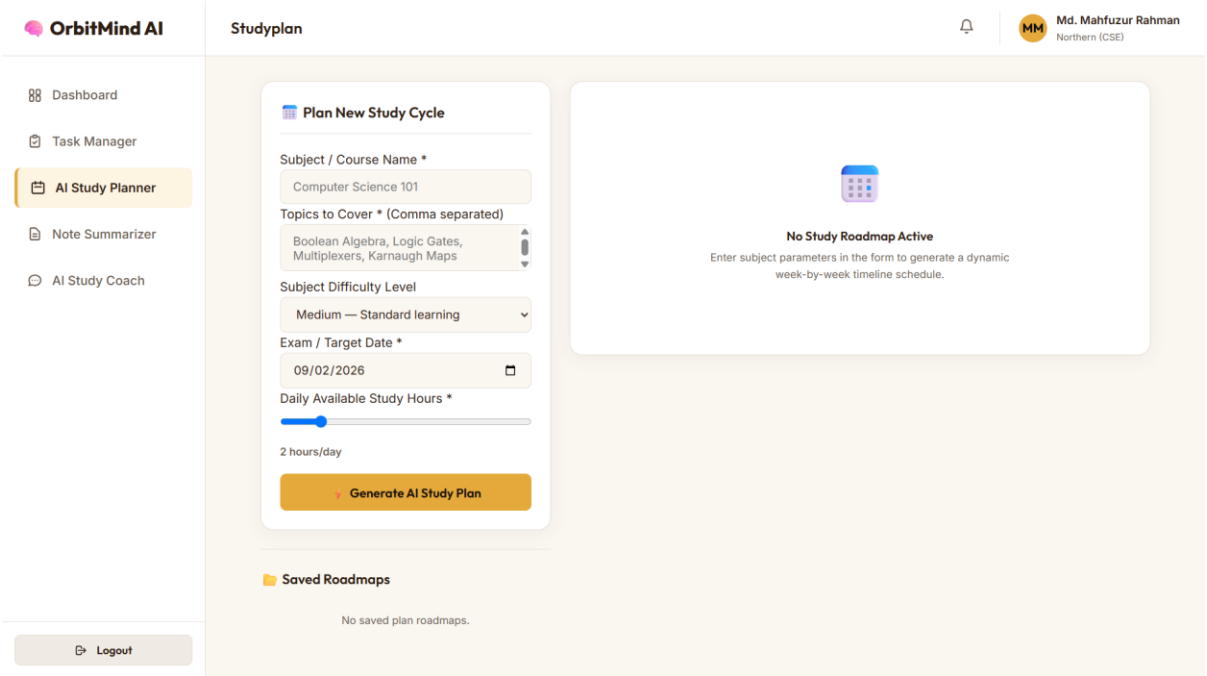


Figure 4: End-to-end AI workflow: User input → Backend prompt assembly → Gemini API response → Database persistence → Frontend rendering.

Mobile/Responsive Interface

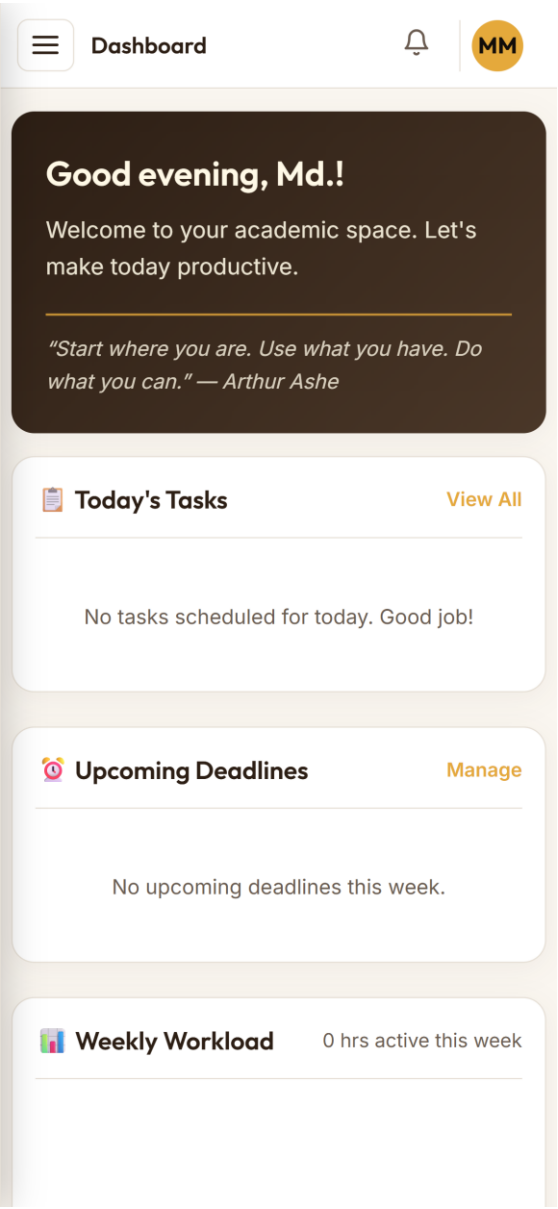


Figure 5: Visual confirmation of responsive, mobile-first UI/UX improvements.

Database Functionality (MongoDB)

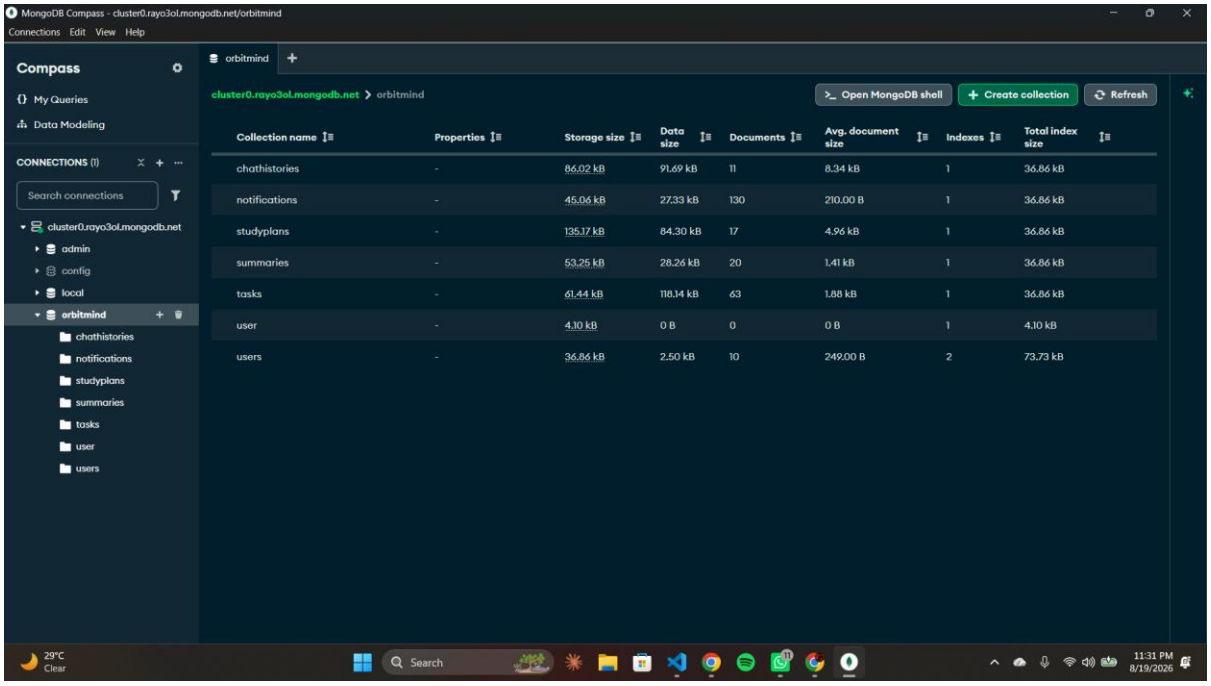


Figure 6: The backend MongoDB Atlas cluster securely storing all application data.

GitHub Repository (Recent Development Activity)

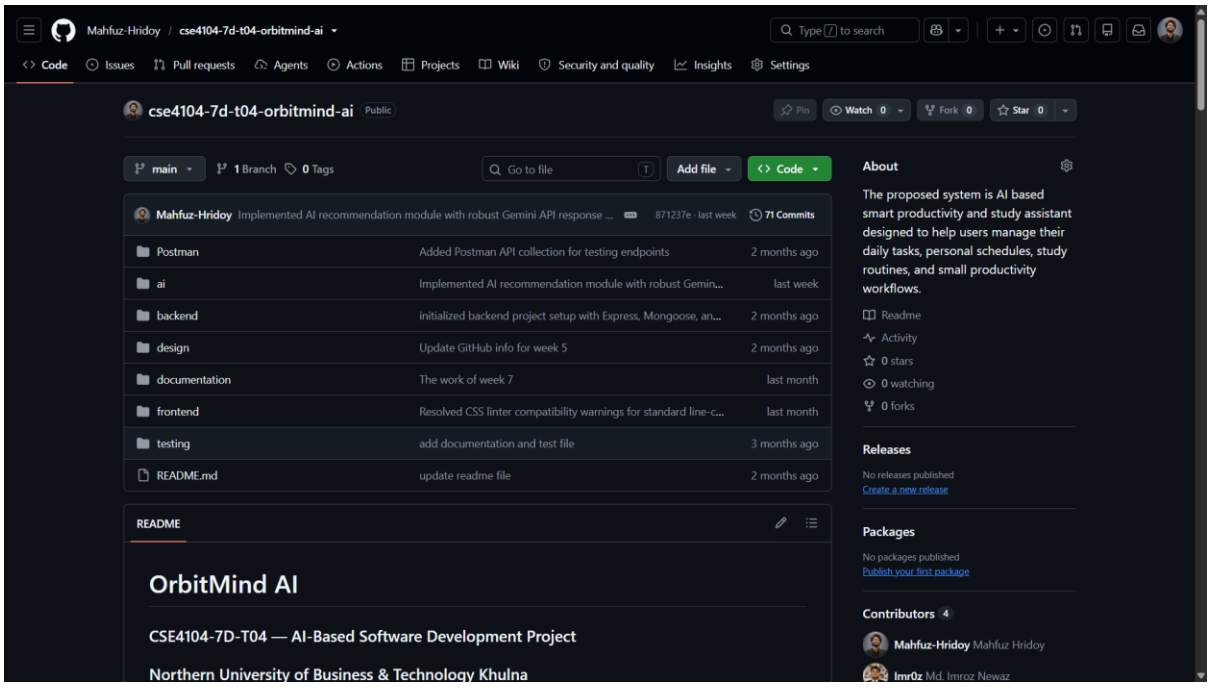


Figure 7: Continuous, meaningful development activity recorded in our version control repository.